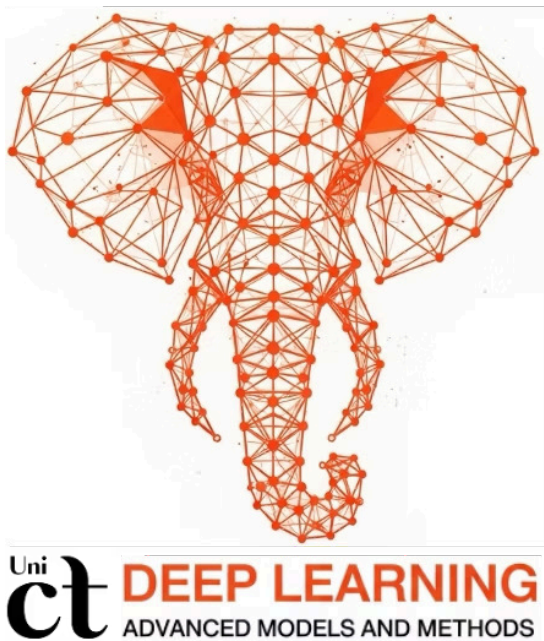




Deep Learning

Advanced Models & Methods

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Laurea Magistrale in Informatica

Dip. di Matematica e Informatica

Università di Catania

Knowledge Distillation

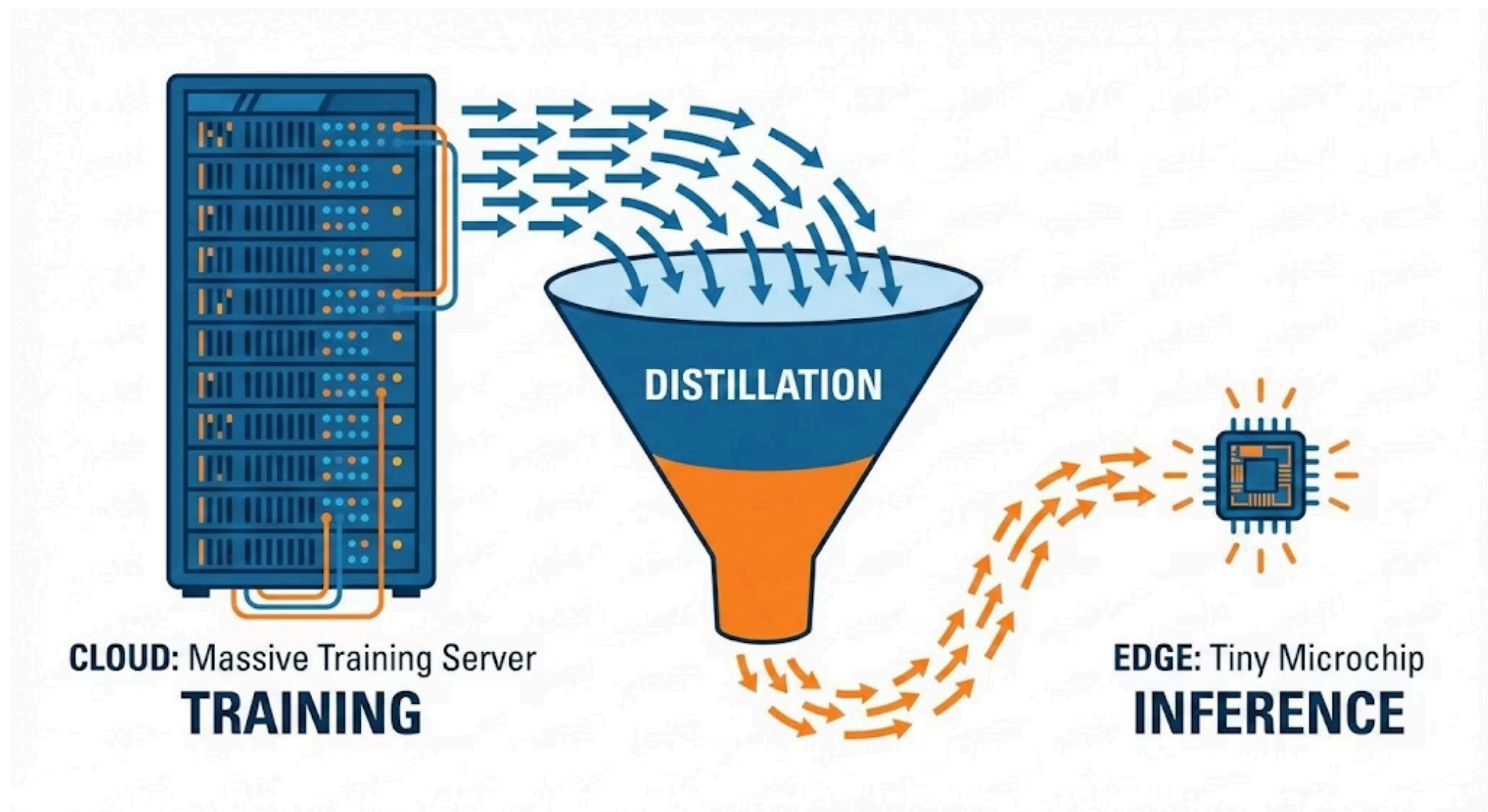
The Inference Gap – Cloud vs. Edge

The Training Reality

Modern deep learning models are **trained on massive cloud infrastructure** featuring NVIDIA A100 or H100 GPUs, with terabytes of memory, unlimited power budgets, and multi-day training windows. This environment enables large-scale models with billions of parameters.

The Deployment Constraint

However, **deployment happens on drastically different hardware**: mobile phones, IoT sensors, autonomous vehicles, and edge devices like Jetson Nano. These systems face strict latency requirements (often sub-100ms), limited memory (megabytes, not gigabytes), and severe power constraints.



- ❑ **KD Philosophy:** Train a compact "Student" model to function effectively in the inference environment by leveraging the rich probability distributions and internal representations learned by a powerful "Teacher" model during training.

The Overparameterization Paradox

Modern deep learning models are often heavily overparameterized. While this capacity is **crucial for navigating complex loss landscapes and finding high-performing solutions**, it frequently leads to networks that contain **significant redundancy once trained**.

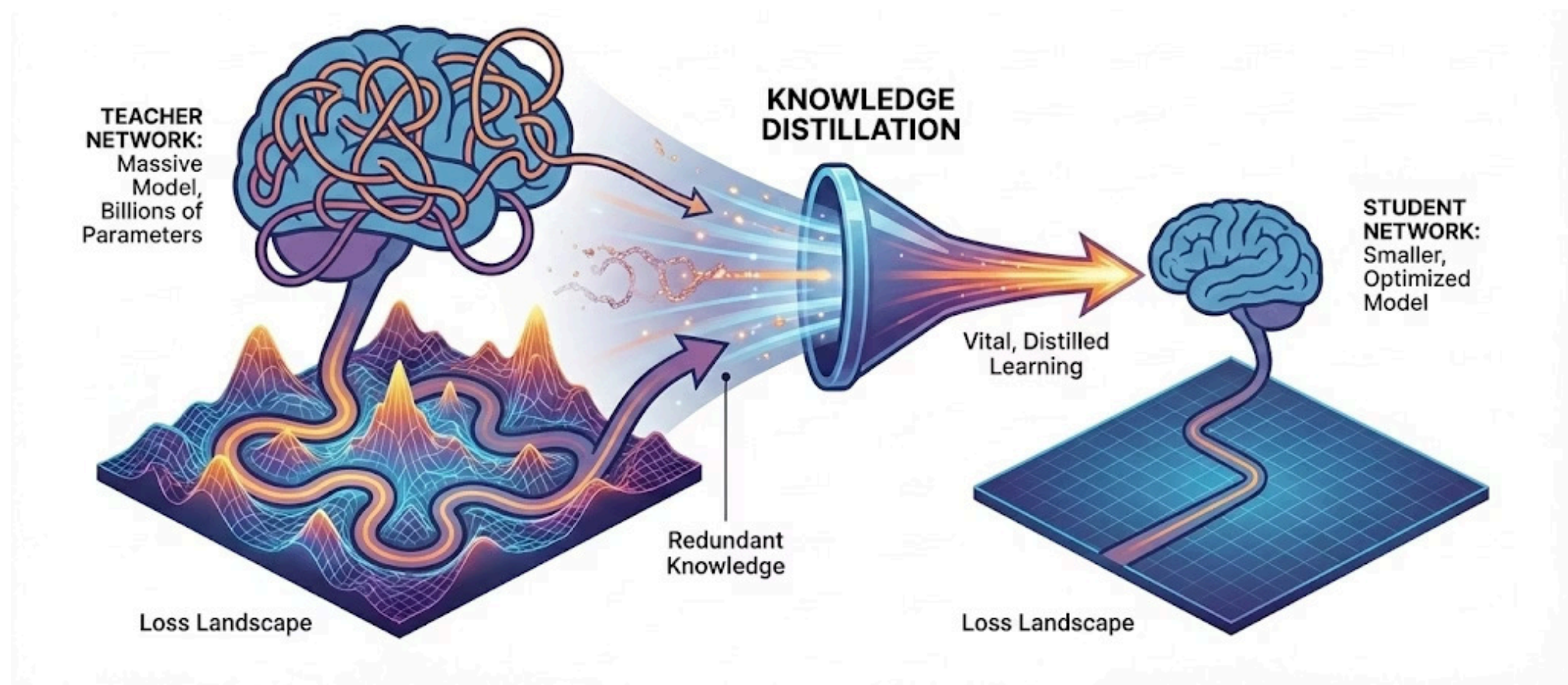
Teacher Network

Massive models with billions of parameters efficiently find solutions but often encode redundant, implicit knowledge.



Student Network

Smaller, optimized networks can learn to express the same core solution efficiently by extracting the teacher's 'essence'.



The main hypothesis is that we can distill a large teacher network into a smaller student network. Here, the large model "shows the way" to the small model.

Model Compression

This technical approach, proposed by Buciluă et al. (2006), outlines a three-step process for distilling a large ensemble's knowledge into a compact neural network. This is a precursor to model Neural Knowledge Distillation.

01

Training the Ensemble

Build a diverse ensemble using methods like bagging, boosting, SVMs, or decision trees. An ensemble contain different models trained independently. It can be used to make predictions on data by averaging individual predictions made by the models.

02

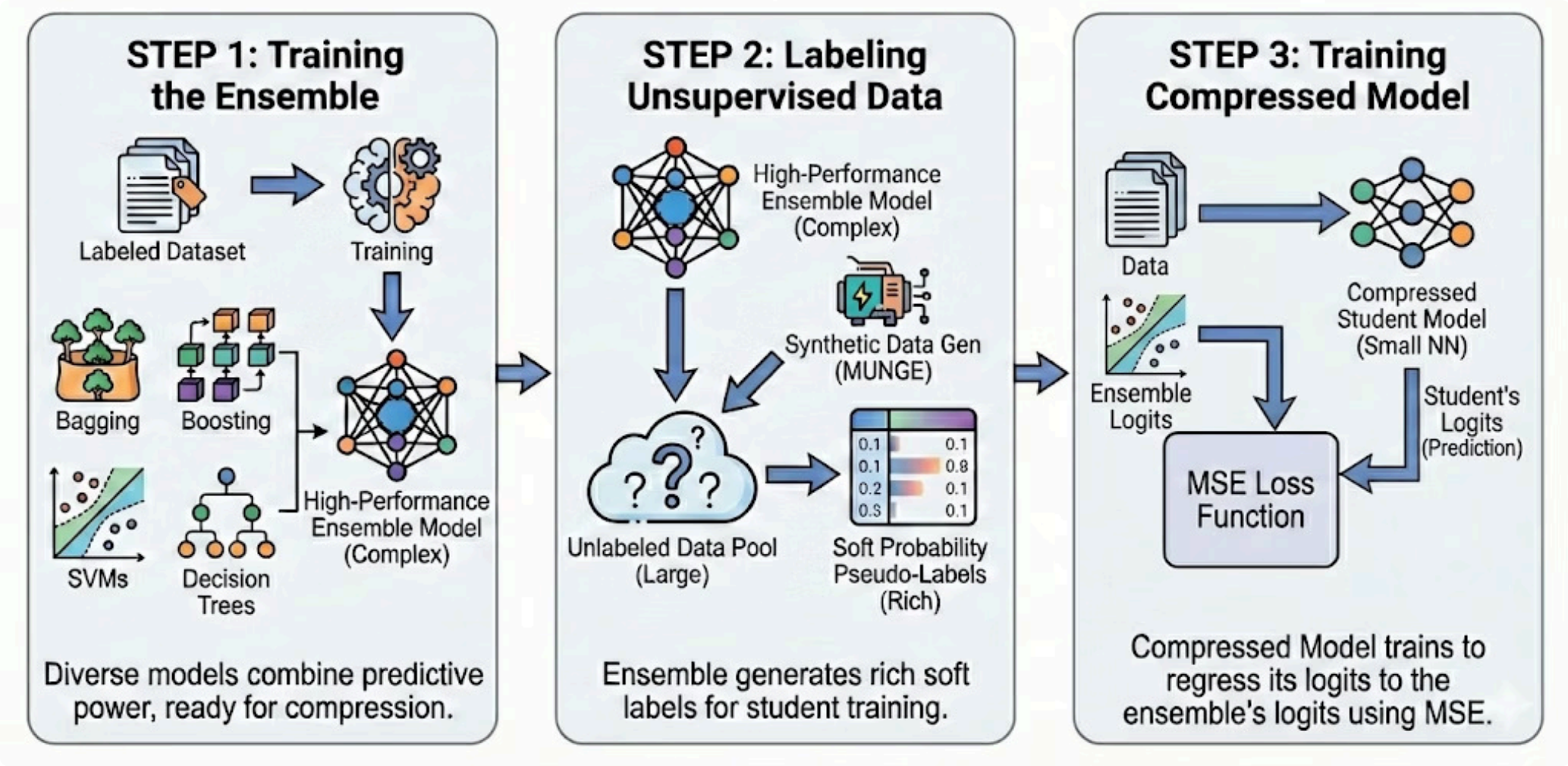
Labeling Unsupervised Data

Leverage the trained ensemble to generate soft probability predictions for a large, unlabeled dataset. These serve as rich pseudo-labels for the student model. If no unlabeled data is available, use synthetic data generation techniques (MUNGE).

03

Training Compressed Model

Train a small neural network (typically 1-3 hidden layers) using the pseudo-labeled data. The loss function is Mean Squared Error (MSE) pushing the small network's prediction with the average logits produced by the ensemble on each example.



This approach achieved remarkable compression, making the student model 1000× smaller and 1000× faster than the original ensemble. Crucially, it retained approximately 97% of the ensemble's performance improvement, proving effective for deployment on resource-constrained devices like mobile phones and PDAs.

Note: We can see the ensemble's logits as Vapnik's privileged information in this case. They encode more than simple labels as they implicitly include uncertainty information.

Buciluă, C., Caruana, R., & Niculescu-Mizil, A. (2006). "Model Compression". KDD 2006.

Learning Using Privileged Information (LUPI)

Vapnik's LUPI Framework

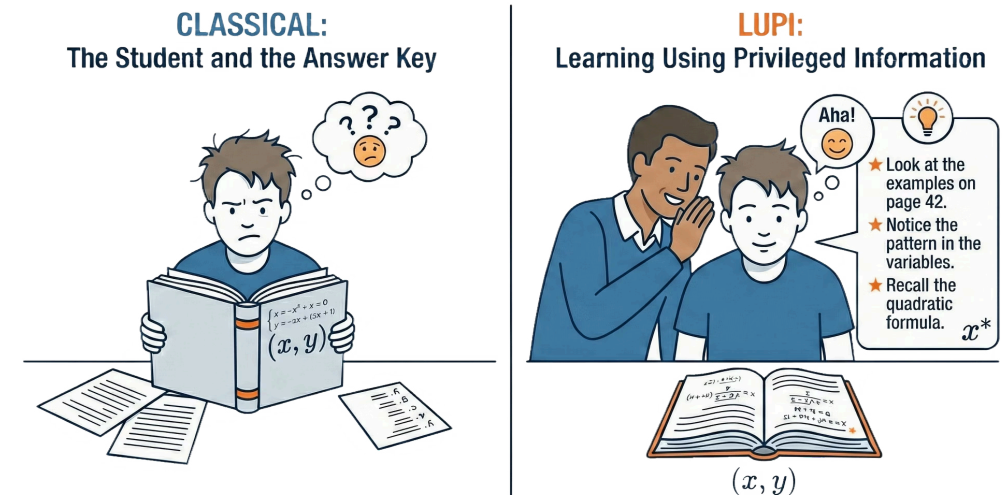
Knowledge Distillation has been proposed in similar forms by Geoffrey Hinton and Vladimir Vapnik in 2015. Vapnik provided a mathematical framework named Learning Using Privileged Information (LUPI).

Classical Learning Paradigm

The student learns from pairs (x, y) where x is the input and y is the ground truth label. The learning rate scales as $O(1/\sqrt{n})$ where n is the number of training samples.

LUPI Paradigm

The student **learns from triplets (x, x^*, y)** , where x^* represents **Privileged Information**—additional context available only during training.

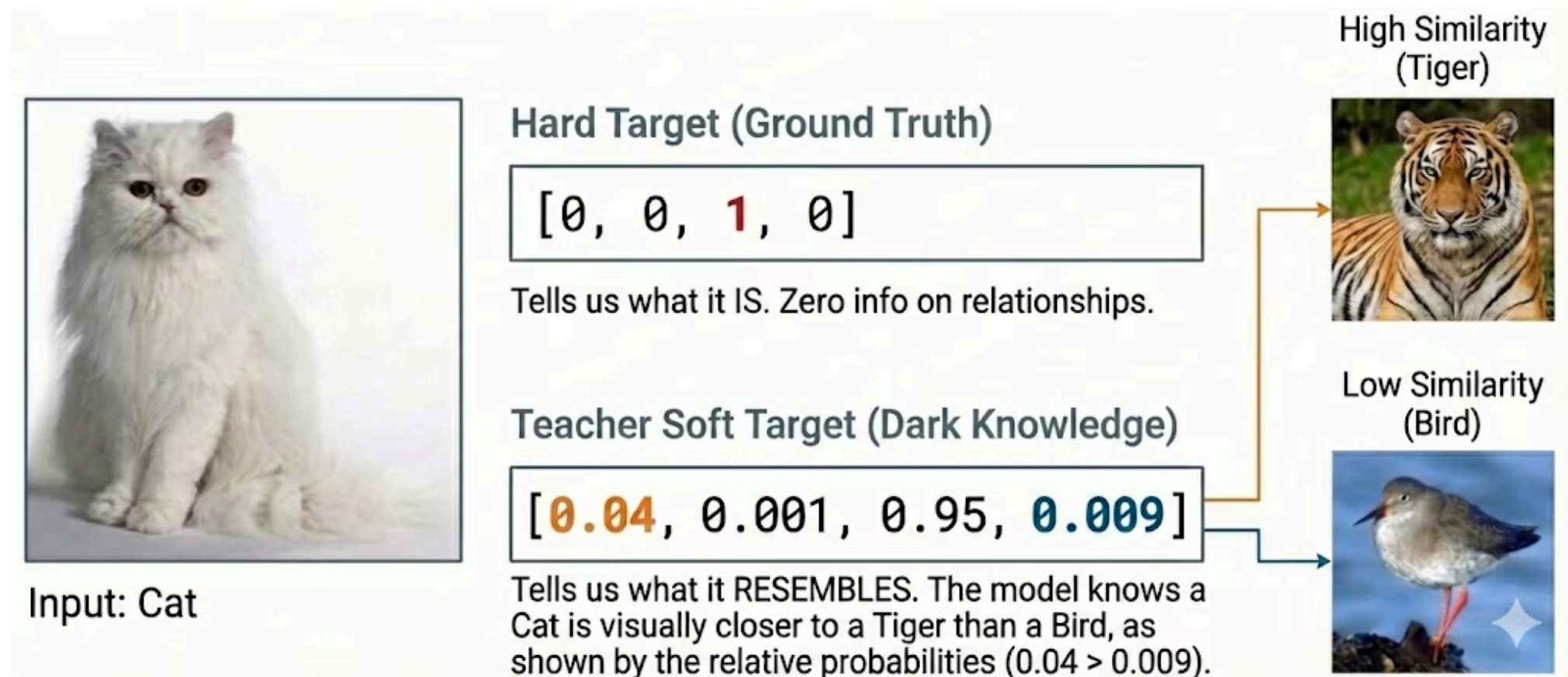


- ❑ **The Acceleration Mechanism:** Vapnik applied this principle to SVM. In this context, the teacher uses the privileged information x^* to provide corrective signals about similarity structure (e.g., providing slack value for examples), fundamentally accelerating the learning rate from $O(1/\sqrt{n})$ to $O(1/n)$. This theoretical speedup explains the empirical success of distillation across diverse domains.

Vapnik, V., & Izmailov, R. (2015). "Learning Using Privileged Information: Similarity Control and Knowledge Transfer". JMLR.

Dark Knowledge

In 2015 Hinton formulates the concept of "dark knowledge" residing into the probability estimates of a trained neural network. While hard labels only encode information on the identity of an example, the predicted probabilities also encode "dark knowledge" on the similarity with negative classes.



The Problem with Hard Labels

Traditional training uses one-hot encoded labels like [0, 1, 0], which does not encode the relative similarity information between classes. The model learns nothing about how similar "dog" is to "wolf" versus "car".

Dark Knowledge Revealed

The key insight: the "tail" of the probability distribution—those small non-zero probabilities for incorrect classes—contains crucial structural knowledge. For example, a model might assign 0.8 to "dog", 0.15 to "wolf", and 0.001 to "car", revealing learned similarity.

Using a learned model to extract dark knowledge

A model trained on the target dataset encodes this "dark knowledge" by encoding uncertainty about predictions. Indeed, for a cat a model may predict small non-zero probabilities for similar classes such as "tiger".

Hinton, G., Vinyals, O., & Dean, J. (2015). "Distilling the Knowledge in a Neural Network". *NeurIPS Workshop*.

The Power of Temperature in Knowledge Distillation

Trained neural networks often produce highly confident predictions, assigning probabilities very close to 1 for the predicted class and near 0 for all others. While good for classification, this extreme certainty hides valuable "dark knowledge" about the subtle relationships between classes. Temperature scaling helps us unveil this hidden information.



Overconfident Predictions

Standard models, after extensive training, output very "sharp" probability distributions, where the correct class gets a near-1 probability and alternatives get near-0. This aggressive confidence masks intricate inter-class similarity.



Softmax's Sharpness

The softmax function, coupled with cross-entropy loss, is designed to maximize the probability of the correct class. This process naturally pushes the logits (raw outputs) for the correct class much higher, leading to highly peaked probability distributions.



Introducing Temperature (T)

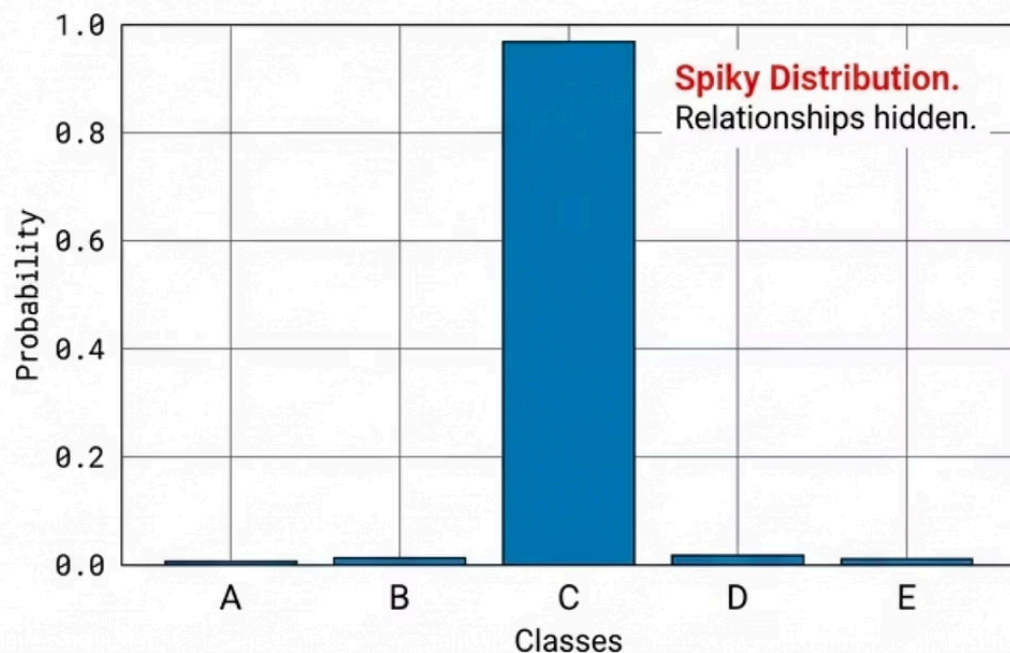
Temperature (T) is a hyperparameter applied to the logits before the softmax function. By dividing the logits by T, we effectively 'soften' the resulting probability distribution, making it less extreme and more informative.



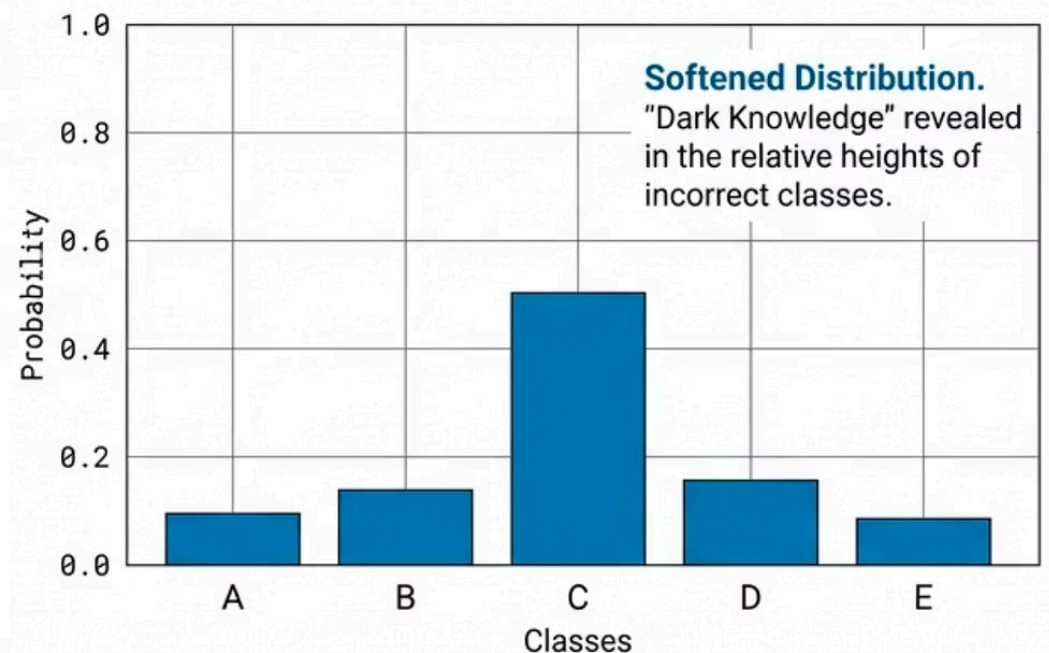
Scaling Logits

When $T > 1$, the logits are scaled down, producing a smoother probability distribution that highlights the small, non-zero probabilities for incorrect but similar classes. A higher T value yields softer probabilities, while $T=1$ reverts to the standard softmax.

Standard Softmax ($T = 1$)



High Temperature ($T = 5$)



Note: this "dark knowledge" can be seen as Vapnik's privileged information, provided by an "intelligent teacher" to a "student" model.

Hinton, G., Vinyals, O., & Dean, J. (2015). "Distilling the Knowledge in a Neural Network". NeurIPS Workshop.

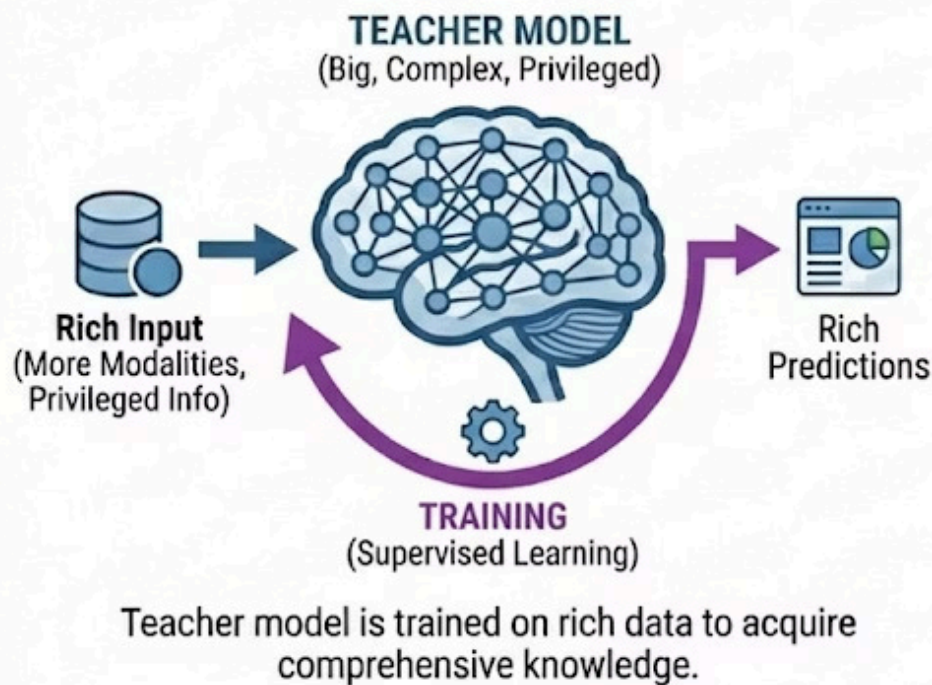
The General Approach



1. Train the Teacher Model

This model is typically large, having access to more modalities or privileged information, capturing deep insights from the data.

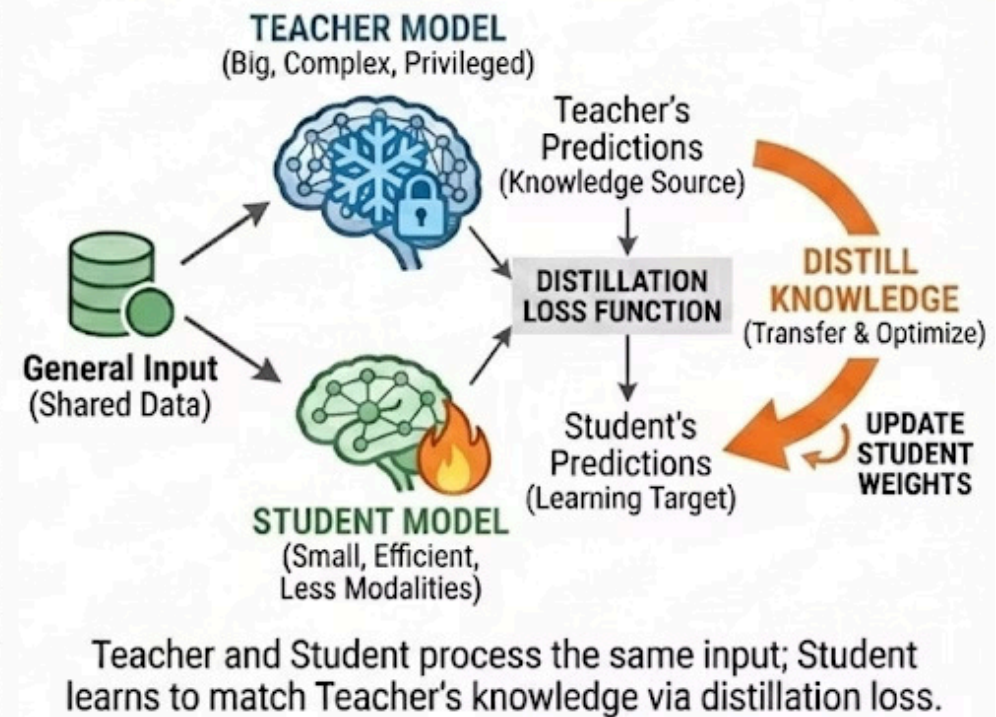
STEP 1: Train the Teacher Model



2. Distill to Student

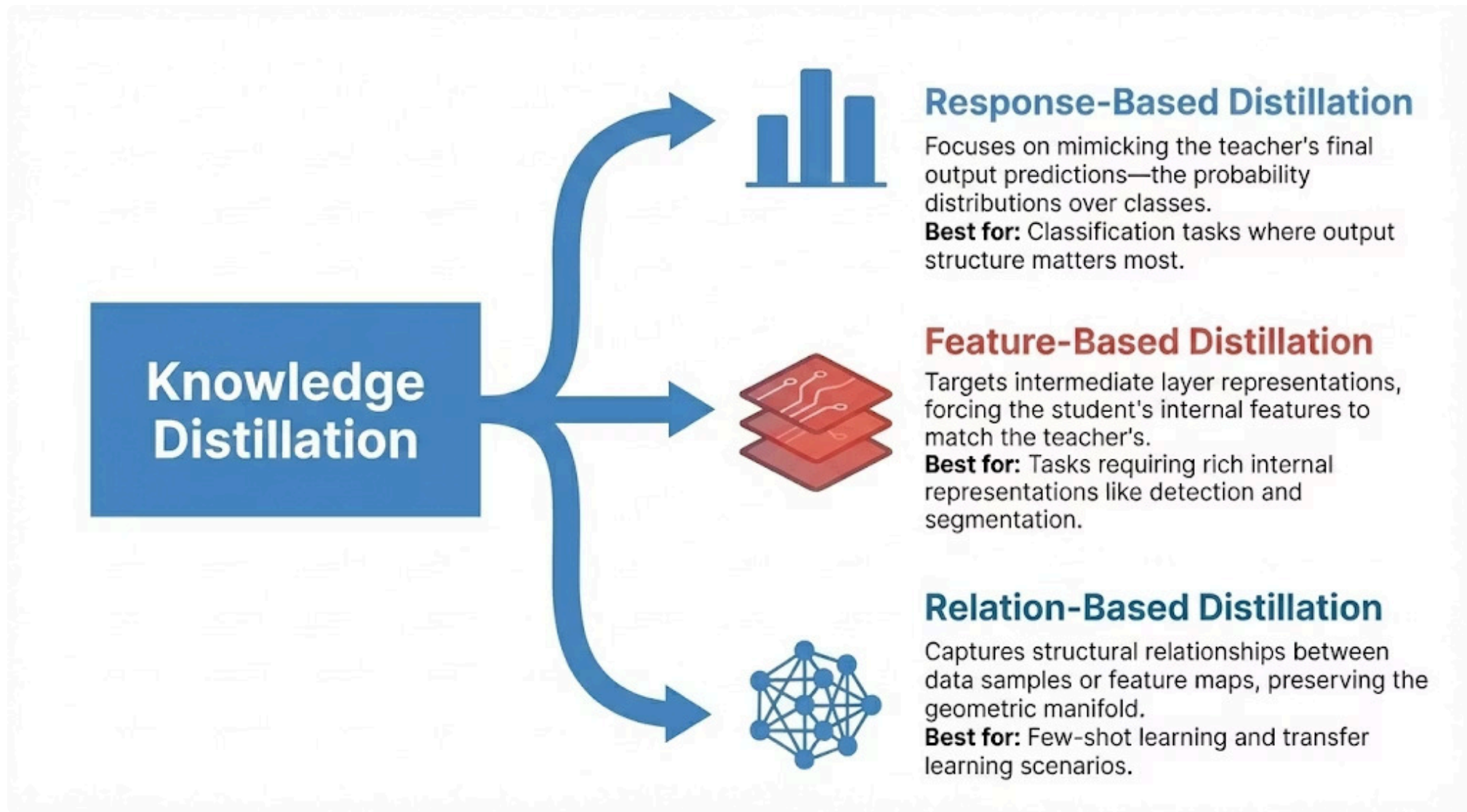
A smaller, more efficient student model is trained. Both teacher and student process the same input in parallel, with knowledge transferred from the teacher to guide the student's learning.

STEP 2: Distill to Student Model



Hinton, G., Vinyals, O., & Dean, J. (2015). "Distilling the Knowledge in a Neural Network". NIPS Workshop.

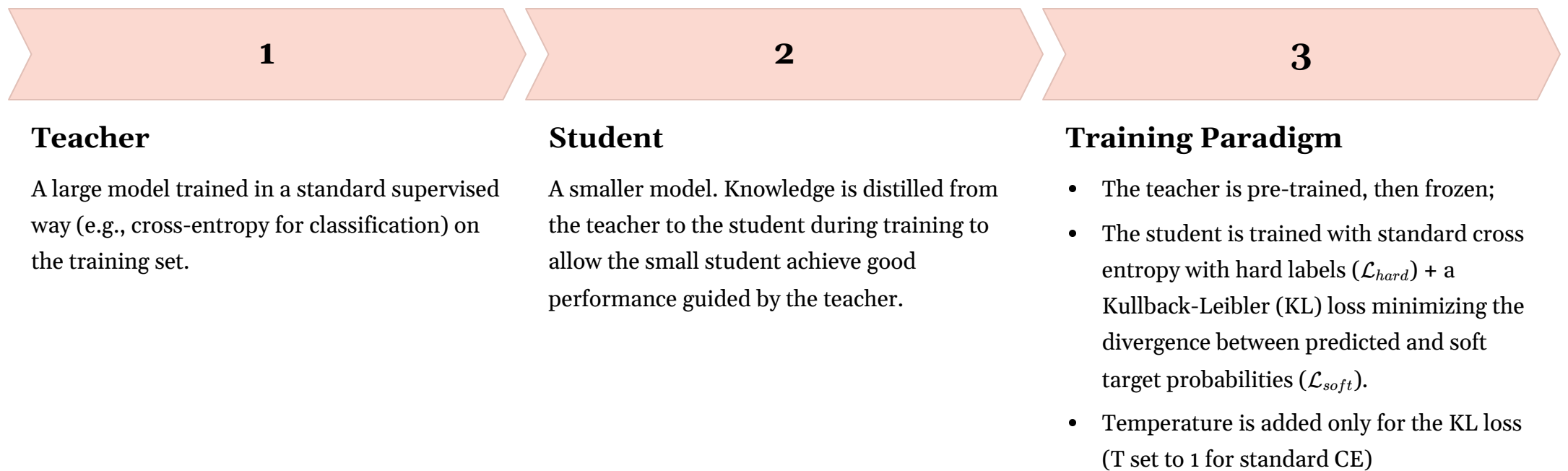
The Taxonomy of Knowledge Distillation



Gou, J., Yu, B., Maybank, S. J., & Tao, D. (2021). "Knowledge Distillation: A Survey". *International Journal of Computer Vision*.

Response-Based – Vanilla Knowledge Distillation

This is Knowledge Distillation approach, as originally introduced by Hinton.

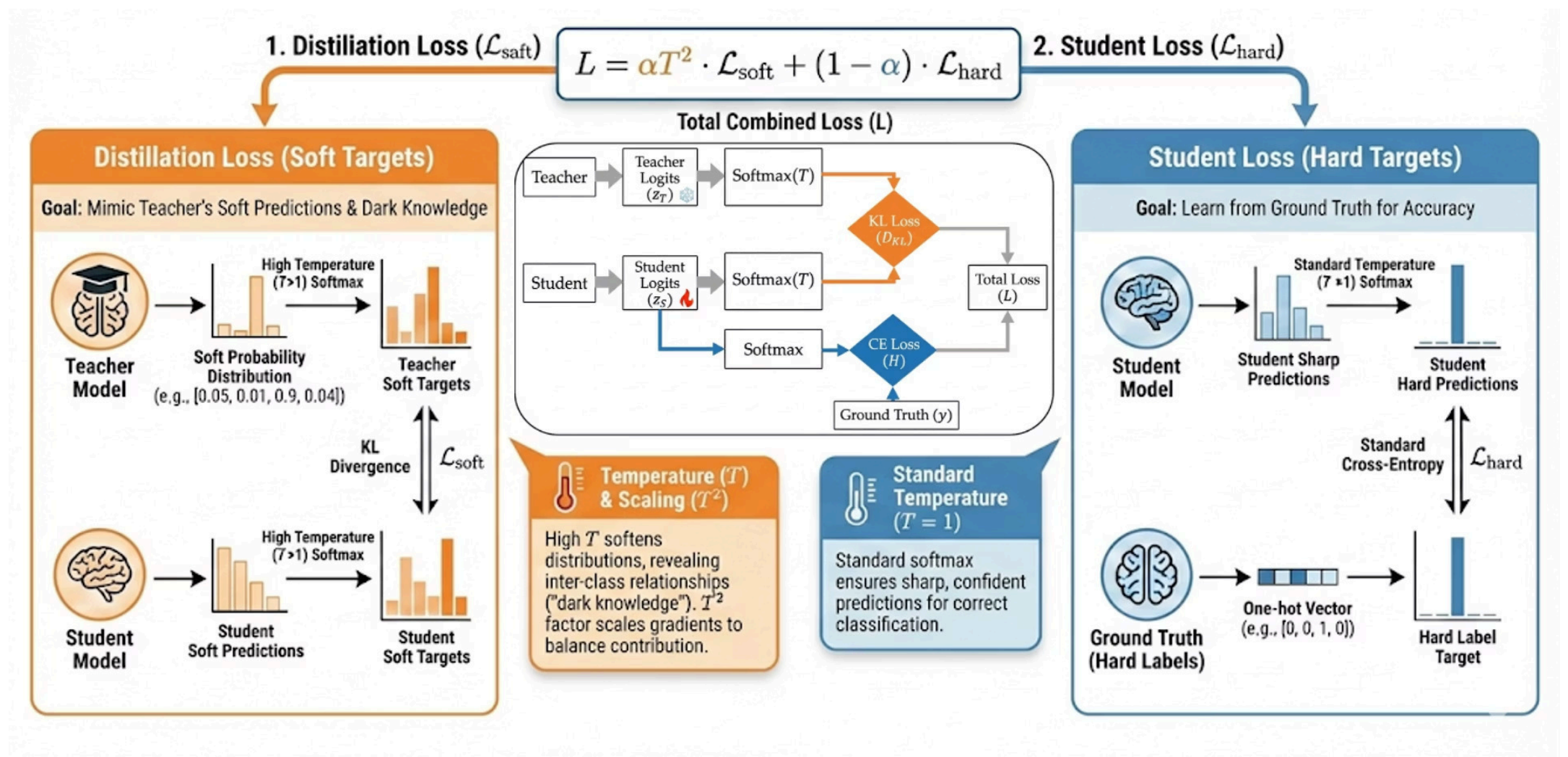


The $\mathcal{L}_{soft}$ loss function is based on the Kullback-Leibler (KL) divergence, which quantifies the cost of using probability q instead of p :

$$D_{KL}(p||q) = \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)}$$

In the context of Knowledge Distillation, q is the probability distribution predicted by the student, while p is the target teacher probability.

The total loss function is described in the following:



Critical Implementation Detail: The KL divergence gradient must be scaled by T^2 to maintain effective learning signals. This scaling compensates for the temperature-induced smoothing of the distribution and ensures gradients remain properly balanced. The parameter α balances the contribution of the two loss functions. Typical values are in the 0.7 - 0.9 range.

Hinton, G., Vinyals, O., & Dean, J. (2015). "Distilling the Knowledge in a Neural Network". NIPS Deep Learning Workshop.

Feature-Based I – FitNets

Output logits capture only the final decision boundary. **The teacher's intermediate layers contain rich hierarchical features**—from edge detectors in early layers to semantic object parts in deep layers. FitNets pioneered transferring this intermediate knowledge through "hint-based training."

01

Select Hint Layers

Choose strategic intermediate layers from the teacher network. Typically the middle layers where semantic features crystallize before the final classification head.

02

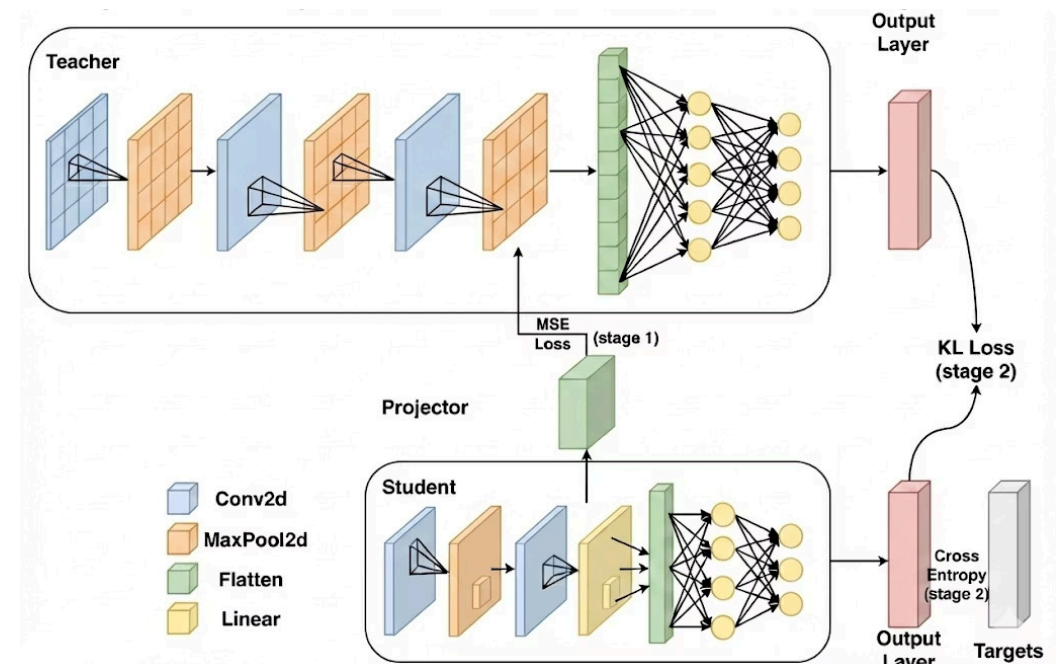
Handle Dimension Mismatch

Teacher and student have different channel dimensions ($CT \neq CS$).
Solution: Insert a learnable 1×1 convolutional "regressor" layer that **projects student features into teacher feature space.**

03

Two-Stage Training

Stage 1: Train the student to match teacher's hint layer using L2 loss.
Stage 2: Fine-tune end-to-end with standard classification loss plus knowledge distillation.



Romero, A., Ballas, N., Kahou, S. E., Chassang, A., Gatta, C., & Bengio, Y. (2015). "FitNets: Hints for Thin Deep Nets". ICLR.

Feature-Based II – Attention Transfer

Focus: Transferring spatial attention patterns—teaching the student *where* to look rather than *what* to see.

The Core Philosophy

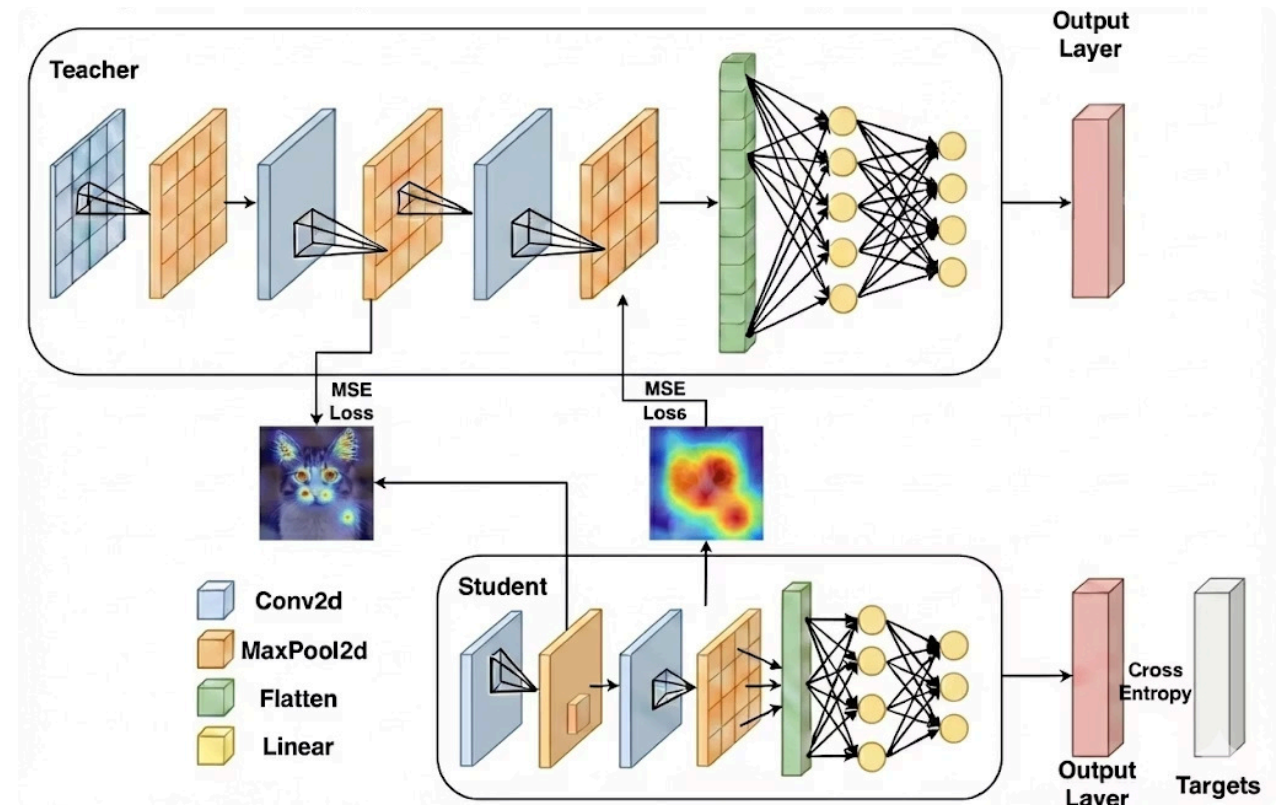
Instead of matching raw feature values (which can be brittle), **Attention Transfer (AT)** matches the **spatial activation patterns**. The key question: *Which regions of the image does the teacher consider important?*

Mechanism

For each feature map F with dimensions $[C, H, W]$, collapse the channel dimension using the L2 norm:

$$A(x) = \sum_c |F_c(x)|^2$$

This produces a 2D attention map showing spatial importance.



The model is trained with the loss:

$$\mathcal{L}_{AT} = \mathcal{L}(\mathbf{W}_S, x) + \frac{\beta}{2} \sum_{j \in \mathcal{I}} \left\| \frac{Q_S^j}{\|Q_S^j\|_2} - \frac{Q_T^j}{\|Q_T^j\|_2} \right\|_2,$$

where $Q_S^j = \text{vec}(F(A_S^j))$ and $Q_T^j = \text{vec}(F(A_T^j))$ are the j -th pair of student and teacher attention maps, L2-normalized and in vectorized form. $\mathcal{L}(\mathbf{W}_S, x)$ is standard cross entropy ($\mathbf{W}_S$ are student's models and x is the input-label pair), β is a hyperparameter regulating the trade-off between the two loss functions.

Zagoruyko, S., & Komodakis, N. (2017). "Paying More Attention to Attention: Improving the Performance of CNNs via Attention Transfer". ICLR.

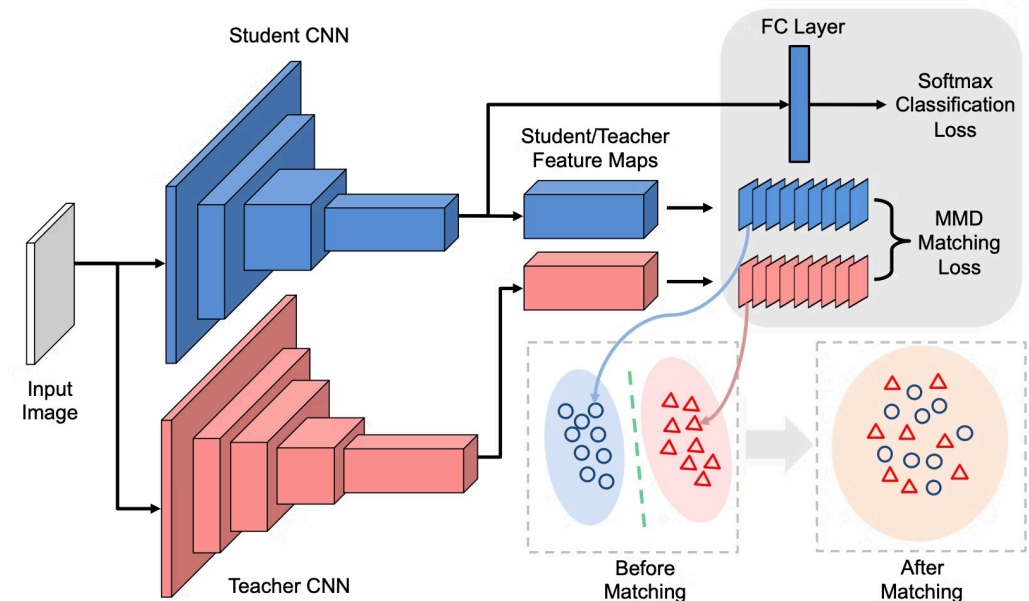
Feature-Based III – Neuron Selectivity Transfer

Building on spatial attention, **Neuron Selectivity Transfer (NST)** moves beyond full feature maps or attention summaries to distill the teacher's **neuron selectivity patterns**—how neurons respond across different spatial regions and samples.

Core Insight: Selectivity Distribution

Instead of rigid feature matching, NST frames knowledge transfer as a **distribution matching problem**. It aligns the statistical properties of teacher and student neuron activations using **Maximum Mean Discrepancy (MMD)**.

- Each neuron's activations across an image form a distribution.
- MMD captures where neurons cluster their activations and how regions are statistically similar.
- This approach is less sensitive to noise and sample density than direct feature matching.



Loss Function: MMD with Polynomial Kernel

NST combines standard cross-entropy ($\mathcal{L}_{hard}$) with an MMD loss term:

$$\mathcal{L}_{NST} = \mathcal{L}_{hard} + \lambda \cdot \text{MMD}^2(F^T, F^S)$$

Where F^T and F^S are the L2 normalized feature maps of a selected layer of the teacher and student respectively. Different kernels can be used for MMD. It is shown that choosing a linear kernel ($K(x, y) = x \cdot y$) is equivalent to attention transfer.

Huang, Z., & Wang, N. (2017). "Like What You Like: Knowledge Distill via Neuron Selectivity Transfer". ICCV.

Relation-Based: Matching the Flow of Solution Procedure (FSP)

The Problem: Direct Feature Matching

FitNet and similar approaches force the student network to match the teacher's intermediate feature maps exactly. This method, while intuitive, proves to be:

- **Too restrictive:** It imposes identical internal representations, limiting the student's ability to discover its own optimal features.
- **Harder to optimize:** The stringent constraints make the training process more difficult and less flexible.

The Core Insight: Learning Solution Flow

What if the teacher didn't just transfer final answers, but rather the **solution procedure itself**? Instead of demanding that the student produces **the same features**, the FSP method proposes to match **how features relate to each other**.

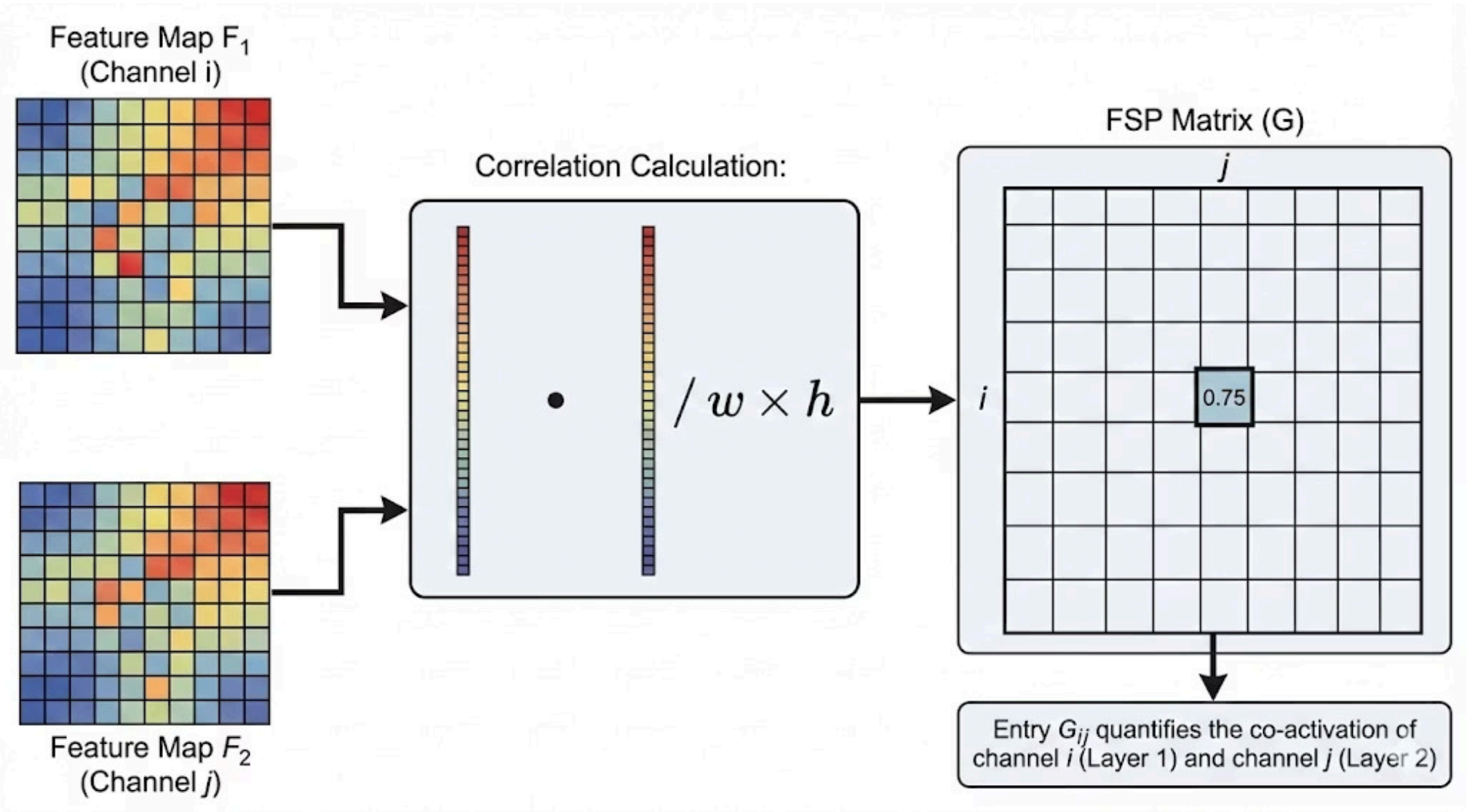
Match the **dynamic interplay** between features, not their static values.

FSP Matrix: Knowledge Representation

The FSP (Flow of Solution Procedure) matrix captures the **relationships between feature channels across consecutive layers**. Given two layers with feature maps $F_1 \in \mathbb{R}^{h \times w \times m}$ and $F_2 \in \mathbb{R}^{h \times w \times n}$, the FSP matrix G is defined as:

$$G_{i,j}(x; W) = \frac{1}{h \times w} \sum_{s=1}^h \sum_{t=1}^w F_1^{s,t,i}(x; W) \cdot F_2^{s,t,j}(x; W)$$

In practice, given two channels i and j , we compute the dot product between feature map i and feature map j and divide by $h \times w$.



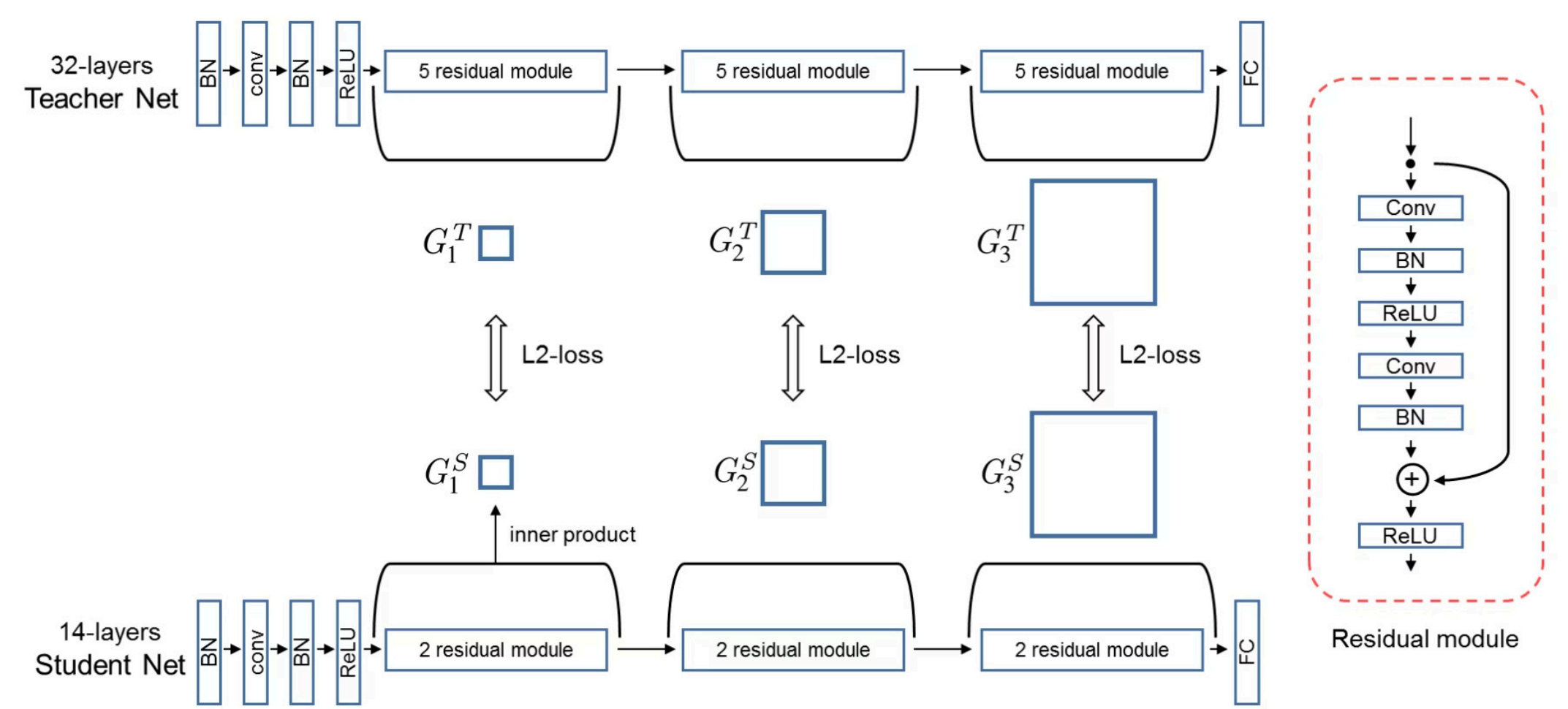
Intuition: The FSP matrix captures directional relationship and co-activation patterns between features. Each entry (i, j) of the FSP matrix quantifies "how much channel i from the first layer and channel j from the second layer activate together." This creates a compact $m \times n$ representation of the teacher's internal processing flow.

FSP Loss Function

Teacher and Student's FSP loss functions are matched with an L2 loss function:

$$\frac{1}{N} \sum_x \sum_{i=1}^n \lambda_i \left\| G^{iT}(x; W_t) - G^{iS}(x; W_s) \right\|_2^2$$

where G^i is the i -th FSP matrix. Layers to be matched are chosen at the beginning and end of residual blocks.



Two-Stage Training

Training happens in two stages:

- The student is distilled from the pre-trained teacher using the L2 loss on FSP matrices;
- The student is fine-tuned using standard cross entropy.

Yim, J., Joo, D., Bae, J., & Kim, J. (2017). A gift from knowledge distillation: Fast optimization, network minimization and transfer learning. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4133-4141).



Other Distillation Paradigms

Cross-Modal & Multimodal Knowledge Distillation

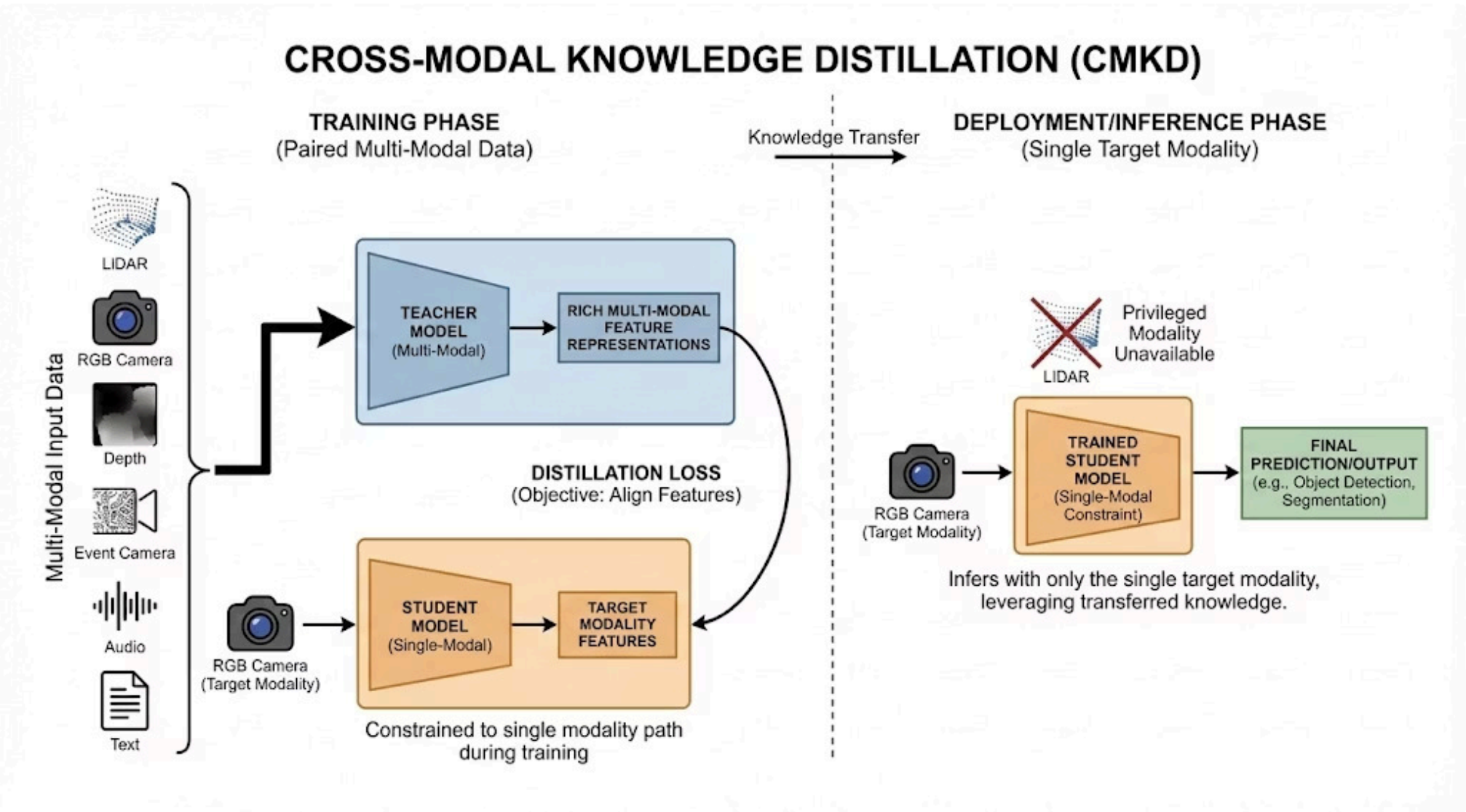
Cross-Modal/Multimodal Knowledge Distillation (CMKD)

Transferring knowledge from a well-annotated source modality to a less-annotated target modality.

The teacher model uses **rich, often privileged, modalities during training**, while the student is constrained to a single, target modality for deployment.

- **Modalities:** RGB, Depth, LiDAR, Event cameras, Text, Audio, Optical Flow
- **Process:** Trained with paired multi-modal data, but infers with only the single target modality.
- **Example:** Teacher trained on LiDAR+Camera data, student deployed with camera-only input.

The teacher may also process unified multimodal representations (e.g., image + text as in CLIP).



Key Applications

			
3D Perception (Autonomous Driving) Distilling LiDAR-based 3D object detection into multi-camera Bird's-Eye-View (BEV) detection for cost-effective systems.	RGB-D Vision Transferring depth estimation capabilities from RGB-D sensors to RGB-only camera systems.	Event-based Vision Enabling high-speed frame capture and motion understanding via efficient event cameras.	Vision-Language Integration Facilitating text-guided image understanding in vision-only models, inspired by models like CLIP.

Deep Mutual Learning – Distillation Without a Teacher

Breaking the Teacher Dependency

Traditional KD requires training a large teacher first, then distilling to a student—a two-stage, computationally expensive process. Deep Mutual Learning (DML) asks: *Can multiple small models teach each other simultaneously?*

The Mechanism

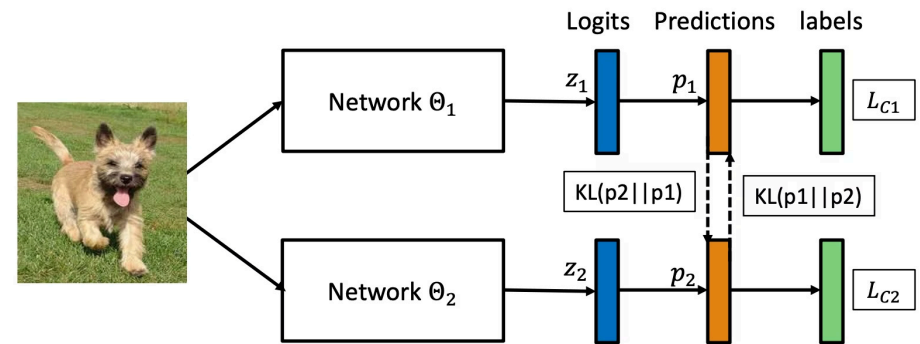
Train two (or more) student networks in parallel from scratch. Each network learns from:

1. Its own predictions on the ground truth (standard supervised learning)
2. The other network's soft predictions (mutual distillation)

Loss for Network A:

$$L_A = CE(y, p_A) + KL(p_B || p_A)$$

Zhang, Y., Xiang, T., Hospedales, T. M., & Lu, H. (2018). "Deep Mutual Learning". CVPR.



Even though both networks start from random initialization, they explore different regions of the solution space due to random weight initialization. This diversity creates complementary knowledge that can be mutually transferred, often resulting in both networks outperforming individually trained models.

Data-Free Knowledge Distillation

GDPR and privacy laws often mandate **data deletion (or limited sharing) after model training**. Medical, financial, and personal data cannot be shared. Yet we still want to distill large models into efficient students. How do we distill without any training data?

This assumes a teacher is given, but the source data is not available anymore.



Generator Initialization

Initialize a generative model (GAN or Diffusion Model) with random weights. This generator will "dream" synthetic data that activates the teacher.



Activation Maximization

Generate synthetic images by maximizing the teacher's class-specific activations. The generator learns to create images that the teacher confidently classifies, even if these images don't look perfectly realistic.



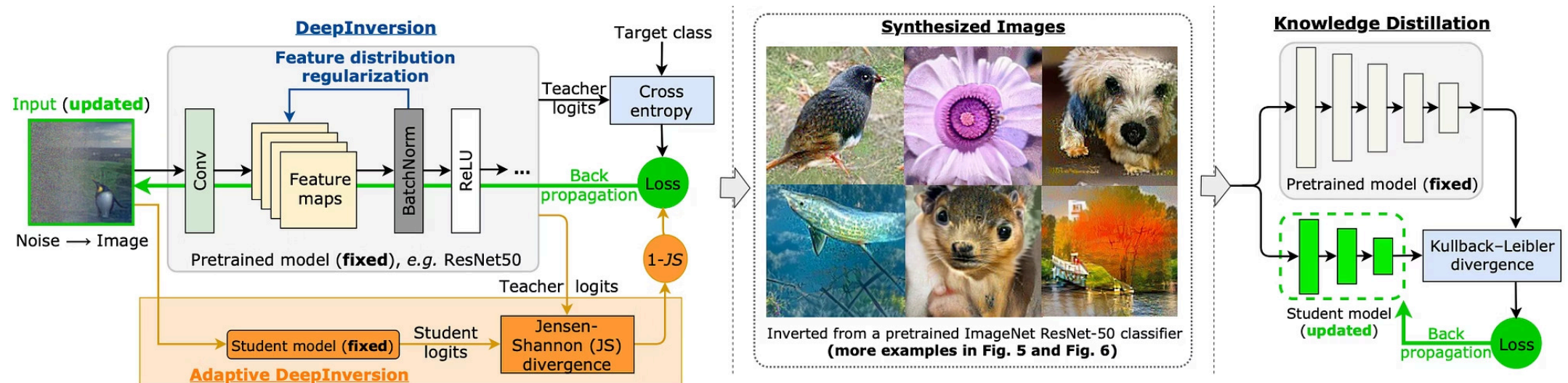
Knowledge Transfer

Use the synthetic images and teacher's soft predictions as pseudo-training data for the student. The student learns the teacher's decision boundaries without ever seeing real data.



Iterative Refinement

Alternate between improving the generator (creating better synthetic data) and training the student (learning from synthetic data) until convergence.



H. Yin et al., "Dreaming to Distill: Data-Free Knowledge Transfer via DeepInversion," CVPR 2020.

G. Fang et al., "Up to 100× Faster Data-free Knowledge Distillation," AAAI 2022.

Xie, et al. (2025). "DiffDFKD: Diffusion-Based Data-Free Knowledge Distillation for Privacy-Preserving Model Compression". CVPR.

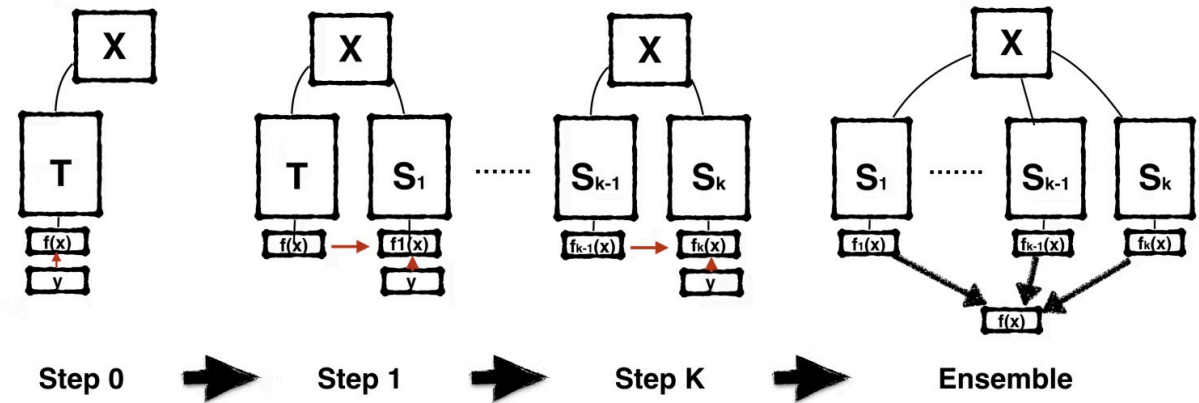
Born-Again Networks – Iterative Self-Improvement

The Counterintuitive Result

Born-Again Networks (BAN) discovered something surprising: using a trained model as a teacher to train an identically-sized student from scratch often *improves* performance.

The Mechanism

1. Train Model A normally on ground truth labels
2. Use Model A as teacher to train Model B (same architecture) using KD
3. Use Model B as teacher to train Model C
4. Repeat for a number of generations
5. Distill the ensemble to a single network



Furlanello, T., Lipton, Z., Tschannen, M., Itti, L., & Anandkumar, A. (2018). "Born-Again Neural Networks". ICML.

Bridging the Gap – Teacher Assistant

The Problem: When the teacher is extremely large (e.g., ResNet152) and the student is extremely small (e.g., MobileNetV2), direct distillation often fails.

The capacity gap is too large—the student cannot capture the teacher's complex decision boundaries, leading to poor knowledge transfer.

Example

Student (Tiny)

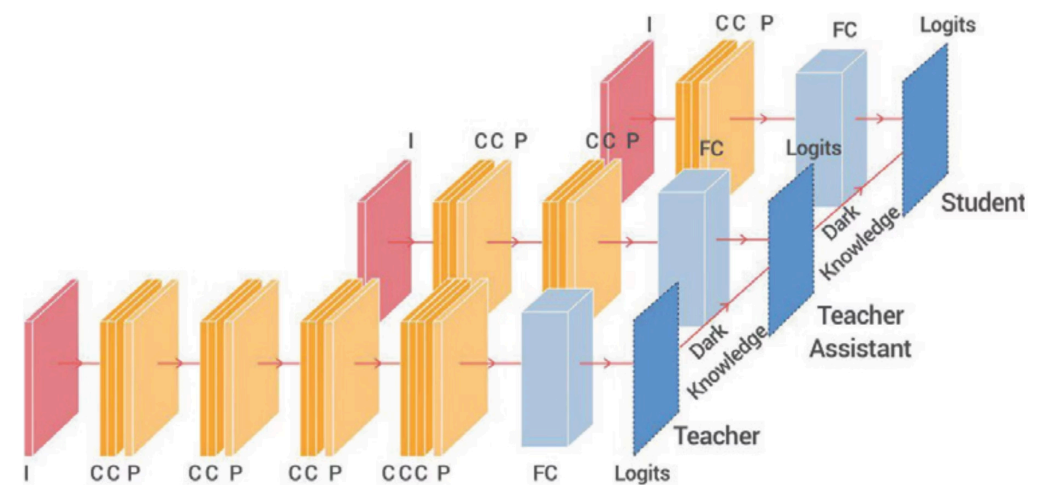
MobileNetV2 (3.5M parameters) - Final deployment target for edge devices

Teacher Assistant (Medium)

ResNet50 (25M parameters) - Intermediate bridge network for progressive distillation

Teacher (Large)

ResNet152 (60M parameters) - Original powerful model with peak performance



Multi-Step Distillation

Instead of Teacher → Student directly, use an intermediate "Teacher Assistant" (TA):

Step 1: Teacher → TA distillation

Step 2: TA → Student distillation

The TA acts as a bridge, making the knowledge more accessible to the student by reducing the representation complexity at each step.

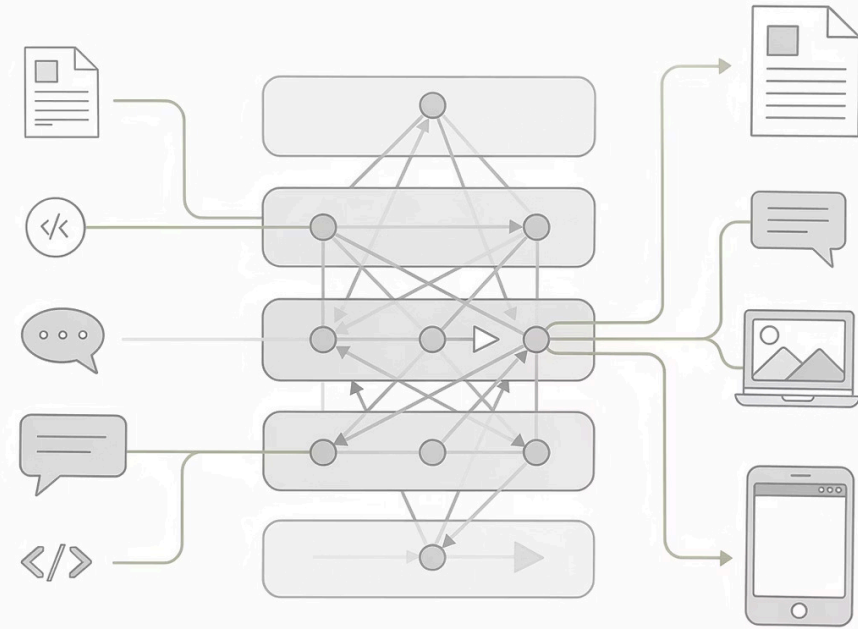
Selecting the TA

The ideal TA should be:

- Medium capacity often near to average accuracy between teacher and student in that architecture family
- Similar architecture family to student for compatibility
- Capable enough to capture teacher's knowledge but simple enough to transfer to student

Mirzadeh, S. I., Farajtabar, M., Li, A., Levine, N., Matsukawa, A., & Ghasemzadeh, H. (2020). "Improved Knowledge Distillation via Teacher Assistant". AAAI.

Applications



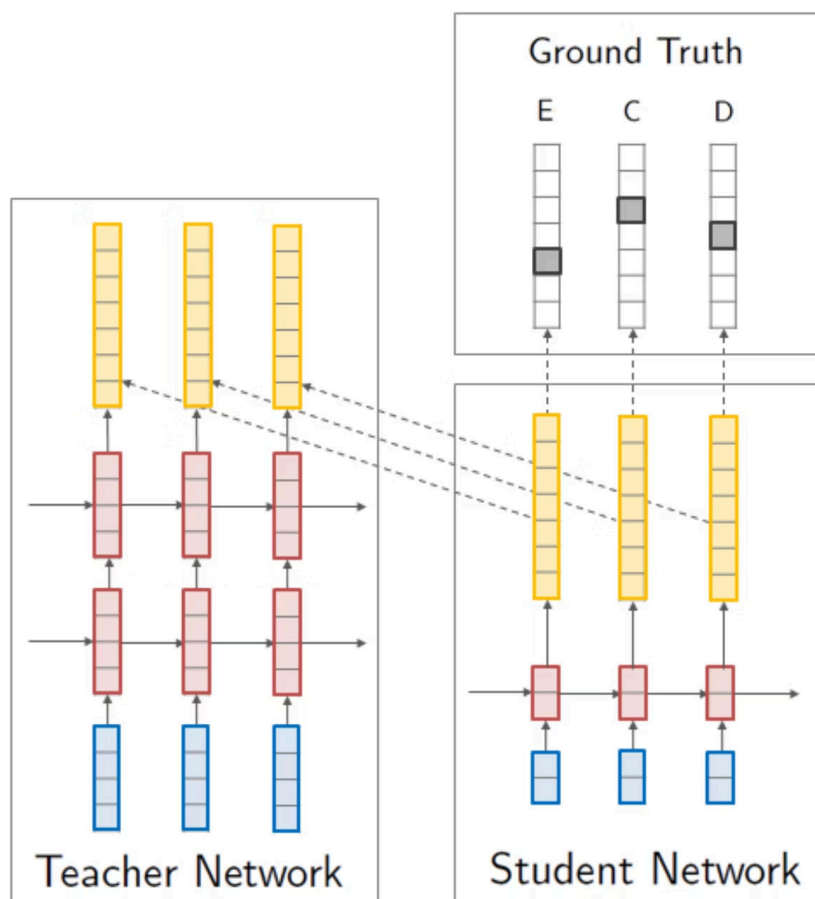
Token-Level vs. Sequence-Level Distillation

Token-Level Distillation (Word-Level KD)

This approach treats sequence generation as a sum of local classification losses at each position. For each token position t , the goal is to minimize the KL divergence (or cross-entropy) between the teacher and student probability distributions for that specific token.

$$L_{\text{token}} = \sum_t \text{KL}(P_T(\cdot | x, y_{<t}) || P_S(\cdot | x, y_{<t}))$$

- Each token distribution still depends on previous tokens $y_{<t}$ through the autoregressive decoder.
- Commonly used in sequence-to-sequence KD and also in BERT-style distillation, but not exclusive to TinyBERT.
- Simple and efficient to implement; works well for many understanding and generation tasks.
- Matches the teacher at the level of per-token distributions, but does not directly model the full sequence distribution $P_T(y_{1:T} | x)$.

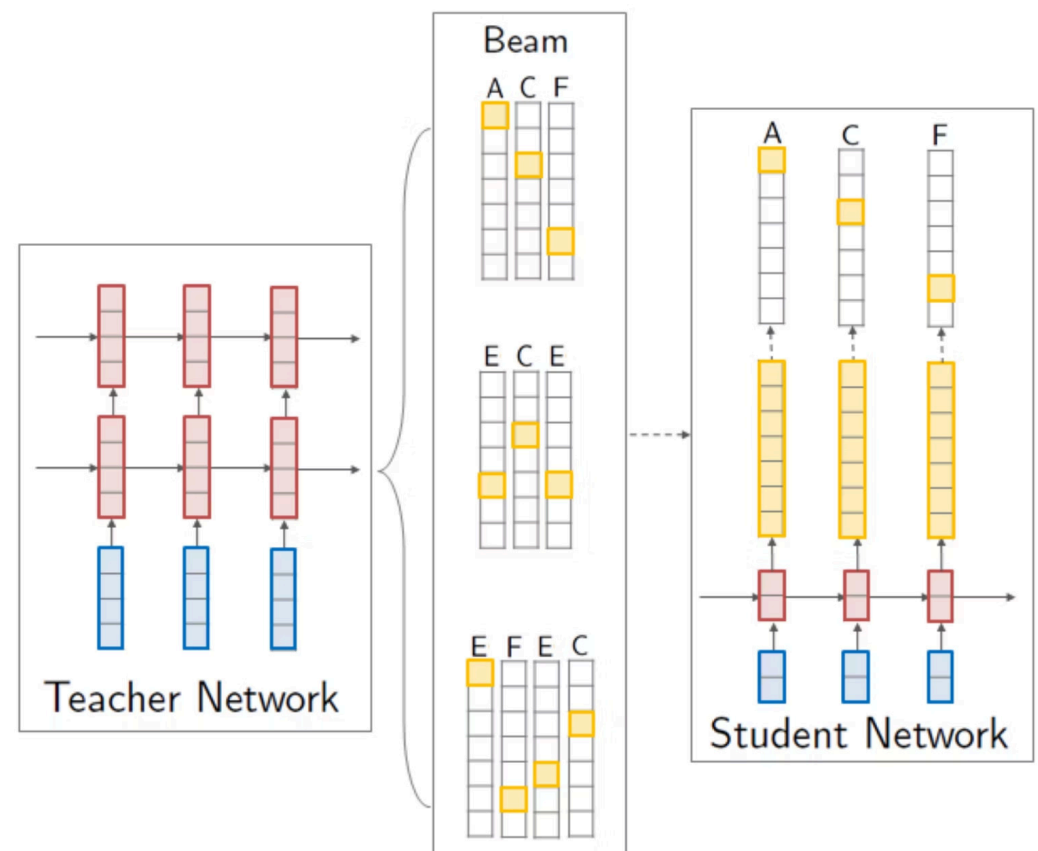


Sequence-Level Distillation

Instead of focusing on individual tokens, this method considers the probability of the entire output sequence as a single unit. It involves using beam search or sampling to generate complete teacher sequences, then training the student to maximize the probability of these generated sequences.

$$y_{1:T}^{(T)} \sim \text{Teacher}(x), \quad L_{\text{seq}} = -\log P_S(y_{1:T}^{(T)} | x)$$

- Often instantiated by taking the teacher's beam-search mode $\hat{y} = \arg \max_y P_T(y | x)$ as a single target sequence.
- Particularly useful for tasks like machine translation, summarization, and dialogue, where global sequence quality and coherence matter.
- Captures inter-token dependencies at the sequence level and can yield more coherent outputs, at the cost of running generation with the teacher.



Kim, Y., & Rush, A. M. (2016). "Sequence-Level Knowledge Distillation". EMNLP.

DeepSeek-R1: Reasoning via Reinforcement Learning

Modern LLMs like DeepSeek-R1 (671B parameters) don't just produce answers—they generate detailed reasoning traces showing step-by-step problem decomposition. Distilling this capability requires more than matching final outputs; we must transfer the entire reasoning trajectory.

Approach: Extracting & Fine-tuning Reasoning Traces

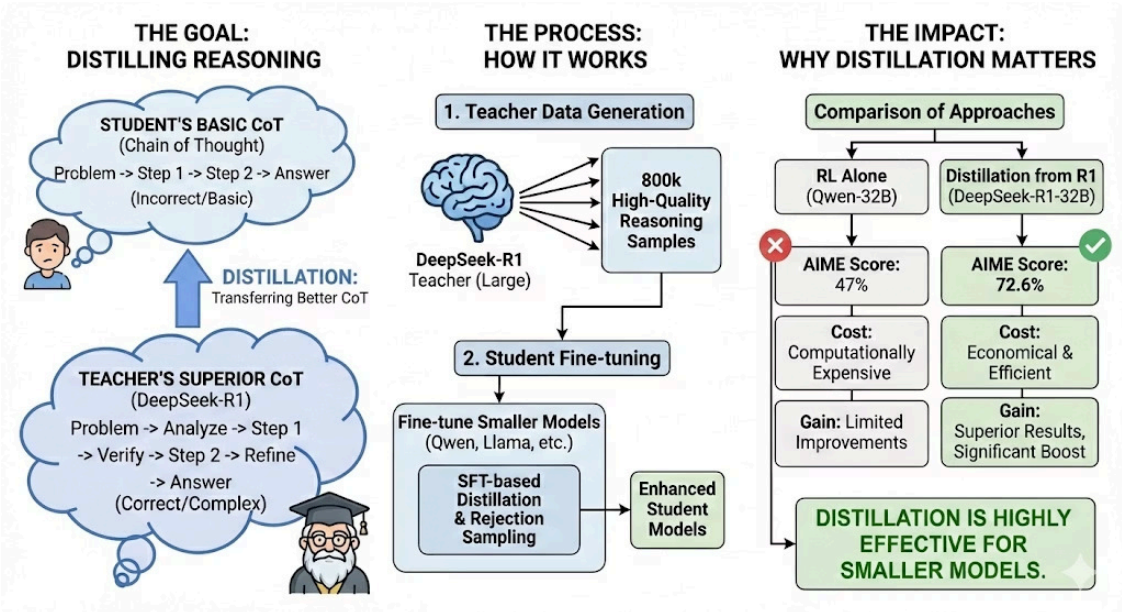
The distillation process involved two key stages:

1. Teacher Data Generation

- Extracted approximately 800,000 high-quality reasoning samples from the powerful DeepSeek-R1 (teacher) model.
- Each sample included detailed intermediate steps and final solutions.

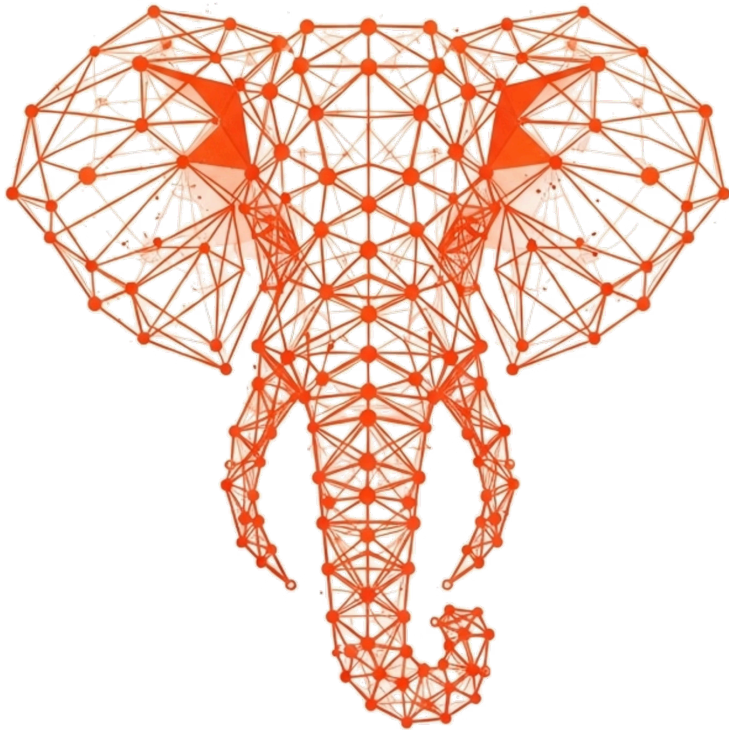
2. Student Fine-tuning

- Fine-tuned smaller base models (e.g., Qwen2.5, Llama family) using these extracted reasoning samples (standard cross-entropy).
- Enhanced data quality via rejection sampling of generated reasoning steps (if not high quality).



Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., ... & He, Y. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Conclusions and Next Steps



We Have Explored:

- **Foundational Theory:** LUPI framework, dark knowledge, and the role of temperature in softening predictions
- **Response-Based Methods:** Vanilla KD
- **Feature-Based Distillation:** FitNets, Attention Transfer, and Neuron Selectivity Transfer for intermediate layer matching
- **Relation-Based Approaches:** FSP matrices
- **Alternative Paradigms:** Deep Mutual Learning, Data-Free KD, Born-Again Networks, and Teacher Assistant
- **LLM Applications:** Transformers, reasoning distillation (DeepSeek)

Uni
ct **DEEP LEARNING**
ADVANCED MODELS AND METHODS

References

1. Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv:1503.02531
2. Romero, A., et al. (2014). FitNets: Hints for thin deep nets. arXiv:1412.6550
3. Zagoruyko, S., & Komodakis, N. (2017). Paying more attention to attention. ICLR
4. Yim, J., et al. (2017). A gift from knowledge distillation: Fast optimization, network minimization and transfer learning. CVPR
5. Park, W., et al. (2019). Relational knowledge distillation. CVPR
6. Tian, Y., et al. (2020). Contrastive representation distillation. ICLR
7. Zhang, L., et al. (2018). Deep mutual learning. CVPR
8. Chen, H., et al. (2019). Data-free learning of student networks. ICCV
9. Furlanello, T., et al. (2018). Born again neural networks. ICML
10. Zhao, B., et al. (2022). Decoupled knowledge distillation. CVPR